Resource Access Protocols

Raihan Uddin
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
raihan.uddin@stud.hshl.de

***Abstract*—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Introduction to Resource Sharing in Real-Time Systems

A new process in real-time system starts share different resources of the system for execution. These resources may vary from data structures, variables, memory (both volatile and non-volatile), files, registers, I/O to processors and others- depending on the task. As stated by [1], "Starting a new process is a heavy job for the OS, which includes allocating memory, creating data structures, and coping code.". In real-time systems, the process of resource sharing is crucial for ensuring that tasks can execute without unnecessary delays or conflicts and thus meeting their deadlines. Many of the embedded systems with 8-bit processors are widely used in products like dishwashers, microwaves, coffee makers and digital watches; it is well established fact that- "Most embedded systems are constrained in terms of processor speed, memory capacity, and user interface." [1]. As stated by [1] also, "Real-time embedded systems are often run in highly resource-constrained environments, which make the system design and performance optimization quite challenging."- thus implying the importance of sharing resources. For allocating resources, required by any task for successful execution, RTOS follows certain protocols.
The protocols can be defined as "a set of rules that govern when and under what conditions each request for resource is granted and how tasks requiring resources are scheduled." [1]. Whereas, it is also mentioned by [2] that, "Any concurrent operating system(OS) must offer protocols to control the access to resources shared (i.e., a piece of code, such as data structures, I/O devices, buffers, and so on) by competing tasks, thus ensuring mutual exclusion in their respective critical sections (a code segment where shared variables can be accessed)". To further emphasize the importance, a well-designed access control protocol can be described as critical rules for preventing deadlocks from occurring in the system [1].

For it's resource constraints and hard deadline requirements, embedded systems often require more stringent resource management compared to general-purpose systems. Hence, "embedded systems must be optimized in terms of size, weight, reliability, performance, cost, and power consumption to fit into the computing environment and perform their tasks" [1]. In general, the RTOS will allocate memory to the process and then loads the module from disk into the memory allocated to it [1]. The process' memory will hold the executable code and initialized data in addition to the uninitialized data and runtime stack into it from the last loaded module with the loader having a default initial size for the stack [1]. Runtime stack can be allocated extra space, not exceeding it's predefined maximum size. [1].

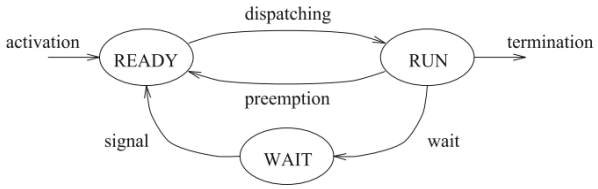

Fig. 1. Waiting state caused by resource constraints [3]

II. Comparison of Resource Sharing Protocols

Competition for resources in real-time systems can lead to deadline anomalies. RTOS offers shared memories to different processes, tasks or threads to share computational data among themselves. However, these memories must be accessed exclusively and this is the reason, as it was mentioned before, that the memory areas are guarded by mutex or semaphores [1].The code segment of the critical section of a task allows it to access the shared data [1].

A. Priority Inversion

Task scheduling is greatly affected by resource sharing and subsequently, when a higher priority task is blocked by a lower priority task- it is known as priority inversion [4].

Priority inversion and deadlocks are major common issues contributing to system failure. Although, there is no protocol that can eliminate priority inversion, the goal of access control is to keep the blocking time of the higher priority tasks under control [1].
For better understanding of how these protocols manage resources allocation among different tasks, a situation can be considered where 3 tasks (J1, J2 and J3) are running in a

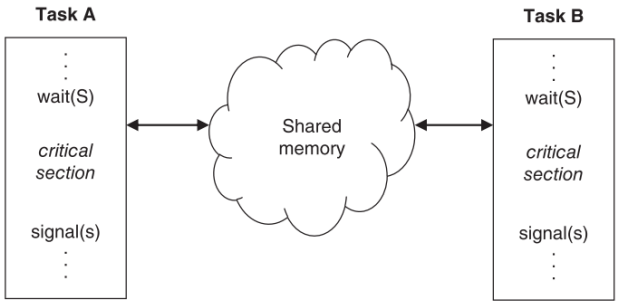

Fig. 2. Shared memory and critical section [1]

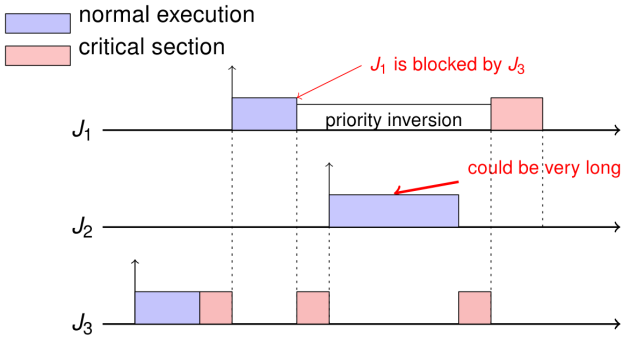

Fig. 3. Priority inversion and its effects [4]

real-time system. There, are m type of resources in a system, namely R1, R2, R3……, Rm where each resource comprises η_i number of indistinguishable units of Ri, i=1, 2, …., m. When a task requests ρ_i units of a resource Ri, request is sent to kernel to lock them and can be denoted as L(Ri, ρ_i). When the resource is longer needed by the task, it is unlocked and can be denoted by U(Ri, ρ_i). As plentiful resources have no effect on scheduling plentiful resources have no effect on scheduling [5], we can avoid considering them here. Also to be considered here- only the serially reusable resources, which are "one that can be used safely by one task at a time and is not depleted by that use" [1]. The examples of these resources are devices, files, databases, and semaphores [1]. The access to resources by every task will be controlled by locking mechanisms.

Binary semaphores- which are also resources, such as mutexes and suspension-based locks, guarantee mutual exclusion [2] by guarding the data shared between several computational tasks [1].Semaphore is described by [3] as "Operating systems typically provide a general synchronization tool, called a semaphore, that can be used by tasks to build critical sections. A semaphore is a kernel data structure that, apart from initialization, can be accessed only through two kernel primitives, usually called wait and signal.". A task which is about to enter a critical section must wait until another task, which is having the required resource, releases it [2]. As stated by [3] that each critical section, operating on a resource R, must start with a wait(W) primitive and end with a signal(S) primitive. All tasks, blocked on a resource with its semaphore, are put in a queue [3]. The overall mechanisms is described by [3] as- "When a running task executes a wait primitive on a locked semaphore, it enters a waiting state, until another task executes a signal primitive that unlocks the semaphore. When a task leaves the waiting state, it does not go in the running state, but in the ready state, so that the CPU can be assigned to the highest-priority task by the scheduling algorithm." [3]. This is how semaphores protect shared resources in real-time systems by barring concurrent access by multiple tasks.

It can be assumed here, for example, that the task J_3 starts at time t_0. Task J_1 arrives in the system at time t_2 and preempts J_3 while it is executing inside its critical section since J_1 has higher priority. Although, after a while at time t_3, when J_1 is attempting to start executing at its critical section, it is being blocked on the semaphore and J_3 will continue, again, to execute at its critical section. At time t_4, yet an another task- 'J_2' can come at the system, which can preempt J_3. Subsequently, the blocking time of J_1 is going to prolong one more time. Here, the maximum blocking time of J_1 does depend not only on the length of the critical section executed by J_3 but also on the worst-case execution time of J_2 [3]. Thus, priority inversion is said to occur from the moment of J_1 being blocked by the semaphore until it once again begins executing. This becomes more complex as any task with intermediate priority can preempt J_3 and indirectly blocks J_1 for an indefinite time. This is why, "the blocking time of a task on a busy resource cannot be bounded by the duration of the critical section executed by the lower priority task" [3]. This unbounded priority inversion incident can cause the task, blocked on resource access, miss its deadline [1].

B. Deadlock

Whereas, another phenomenon- deadlock occurs when two or more tasks are waiting for resources to be released by one another, thus creating a cycle of dependencies and situation where none of them can proceed. Deadlock occurs when the following four conditions coincide [1]:

- Mutual exclusion: At least one resource must be held in an exclusive mode.
- Hold and wait: A task will hold a resource while also waiting for another resource.
- No preemption: A resource cannot be forcibly taken from a task holding it.
- Circular wait: There must be a set or tasks where each task waits for a resource held by the next task in the set- creating a circular-chain like dependency issue.

Let it be considered that there are two tasks- T_H and T_L in a system, where T_L starts executing first. After a short while, it starts using resource A by blocking it. Then, T_H arrives at the system and preempts T_L, as T_H has the higher priority. Later, it continues running by taking the resource B, until attempting to lock the resource A. As A is held by T_L, T_H gets blocked. Similary, T_L, sometime later, when tries

to block resource B, it gets blocked. Now both gets blocked from sharing resources as they are held by each other. No task can make progress in this situation. As it can be seen

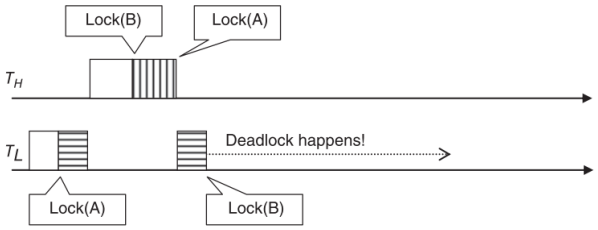

Fig. 4. Deadlock [1]

that resource sharing can cause serious operational problems in real-time systems, there must have to be set of rules to regulate the access to resources by competing tasks. There have been several well-known resource access protocols in place to handle priority inversion and deadlocks efficiently [1]. According to [3], the most common protocols are:

i. Non-Preemptive Protocol (NPP)
ii. Highest Locker Priority (HLP), also called Immediate Priority Ceiling (IPC);
iii. Priority Inheritance Protocol (PIP)
iv. Priority Ceiling Protocol (PCP)
v. Stack Resource Policy (SRP)

III. Classic Protocols to Handle Resource Sharing

It is stated by [3] that the first four protocols, which are mentioned above, have been developed fixed priority assignments but some of them have also been extended under EDF; whereas, the last one is applicale for both fixed and dynamic priority assignments.

References

[1] Jiacun Wang, *Real-Time Embedded Systems*, ser. Quantitative Software Engineering Series. 111 River Street, Hoboken, NJ 07030, USA: John Wiley & Sons, Inc., 2017.
[2] L. M. dos Santos, G. Gracioli, T. Kloda, and M. Caccamo, "Supporting single and multi-core resource access protocols on object-oriented rtoses," vol. 27, pp. 31–50, 2023.
[3] G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed. Springer Nature, 2024.
[4] Jan Reineke, "Verification of real-time systems resource sharing: Advanced lecture, summer 2015," 2015. [Online]. Available: https://embedded.cs.uni-saarland.de/lectures/realtimesystems15/resourceSharing.pdf
[5] "Resource access control in real-time systems: Advanced operating systems (m)." [Online]. Available: https://csperkins.org/teaching/2011-2012/adv-os/lecture08.pdf